

Team Notebook

October 18, 2024

Contents

Contents		15 Formulas	4	31 Inverse Modulo	8	
1	Array Initializaton	2	16 Graph BFS	4	32 Least Common Multiple - LCM	8
2	Base Conversion 2	2	17 Graph Bridges and Articulation Points	4	33 Modulo Multiplication	8
3	Base Conversion	2	18 Graph DFS	5	34 Primality Check	9
4	Binary Exponential	2	19 Graph Dijkstra	5	35 Primes	9
5	Binary Index Tree	2	20 Graph Eulerian Circuit	5	36 Quicksort	9
6	Binary Search	2	21 Graph Flood Fill	5	37 Segment Trees Lazy	9
7	Binomial Coefficient DP	2	22 Graph Flow-Cut, Ford-Fulkerson	5	38 Segment Trees	9
8	Bit Manipulation	3	23 Graph Floyd Cycles	6	39 Template	10
9	Convex Hull - Monotone Chain	3	24 Graph Floyd-Warshall	6	40 Testcase Generator	10
10	DP Longest Common Increasing Subsequence - LCIS	3	25 Graph Paths in a Grid	6	41 Tree DFS	10
11	DP Longest Common Subsequence - LCS	3	26 Graph Prim - MST	6	42 Tree Depth Width	10
12	DP Longest Increasing Subsequence - LIS	4	27 Graph Representation	7	43 Tree Traversal	10
13	Efficiencies	4	28 Graph SPFA - Better Bellman-Ford	8	44 Tries	11
14	Exponential mod	4	29 Graph Topological Sorting - DFS	8	45 Union Find	11
			30 Greatest Common Divisor - GCD	8	46 cmath Tricks	11

1 Array Initializaton

```
// for 1d array, use STL fill_n or fill to initialize array
fill(a, a+size_of_a, value)
fill_n(a, size_of_a, value)
// for 2d array, if want to fill in 0 or -1
memset(a, 0, sizeof(a));
// otherwise, use a loop of fill or fill_n through every a[i]
fill(a[i], a[i]+size_of_ai, value) // from 0 to number of row.
```

2 Base Conversion 2

```
// Base conversion from base M (up to 16) to base 10
string num="0123456789ABCDE";
int mToTen(string n, int m)
{
    int multi=1;
    int result=0;
    for (int i=n.size()-1;i>=0;i--)
    {
        result+=num.find(n[i])*multi;
        multi*=m;
    }
    return result;
}
```

3 Base Conversion

```
// Convert number from base 10 to base M (up to 16)
char a[16]={ 0 , 1 , 2 , 3 , 4 , 5 , 6
            , 7 , 8 , 9 ,
            A , B , C , D , E , F };
string tenToM(int n, int m)
{
    int temp=n;
    string result="";
    while (temp!=0)
    {
        result=a[temp%m]+result;
        temp/=m;
    }
    return result;
}
```

4 Binary Exponential

```
int binpow (int a, int n)
{
    int res = 1;
    while (n)
        if (n & 1)
            res *= a;
        --n;
    }else
    {
        a *= a;
        n >>= 1;
    }
    return res;
}
```

5 Binary Index Tree

```
struct binary_index_tree {
    int n;
    int t[2 * N];
    void add(int where, long long what){
        for (where++; where <= n; where += where & -where){
            t[where] += what;
        }
    }
    void add(int from, int to, long long what) {
        add(from, what);
        add(to + 1, -what);
    }
    long long query(int where){
        long long sum = t[0];
        for (where++; where > 0; where -= where & -where){
            sum += t[where];
        }
        return sum;
    }
};
```

6 Binary Search

```
// Method 1:
int a = 0, b = n-1;
```

```
while (a <= b) {
    int k = (a+b)/2;
    if (array[k] == x) {
        // x found at index k
    }
    if (array[k] > x) b = k-1;
    else a = k+1;
}

// Method 2:
int k = 0;
for (int b = n/2; b >= 1; b /= 2) {
    while (k+b < n && array[k+b] <= x) k += b;
}
if (array[k] == x) {
    // x found at index k
}
```

```
// STL:
// lower_bound() = pointer to first element at least x
// upper_bound() = pointer to first element larger than x
// equal_range() = both of the above
```

```
auto k = lower_bound(array,array+n,x)-array;
if (k < n && array[k] == x) {
    // x found at index k
}
```

```
// Count Elements with value x:
auto a = lower_bound(array, array+n, x);
auto b = upper_bound(array, array+n, x);
cout << b-a << "\n";
// Same but shorter
auto r = equal_range(array, array+n, x);
cout << r.second-r.first << "\n";
```

7 Binomial Coefficient DP

```
#define MAXN 100 // largest n or m
long binomial_coefficient(int n, int m) // compute n choose m
{
    int i,j;
    long bc[MAXN][MAXN];
    for (i=0; i<=n; i++) bc[i][0] = 1;
    for (j=0; j<=n; j++) bc[j][j] = 1;
    for (i=1; i<=n; i++)
        for (j=1; j<i; j++)
```

```

        bc[i][j] = bc[i-1][j-1] + bc[i-1][j];
    return bc[n][m];
}

```

8 Bit Manipulation

```

// g++ Functions
// __builtin_clz(x): the number of zeros at the beginning of
// the number
// __builtin_ctz(x): the number of zeros at the end of the
// number
// __builtin_popcount(x): the number of ones in the number
// __builtin_parity(x): the parity (even or odd) of the
// number of ones
// (for long long versions use suffix ll)

int x = 5328; // 00000000000000000001010011010000
cout << __builtin_clz(x) << "\n"; // 19
cout << __builtin_ctz(x) << "\n"; // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n"; // 1

```

9 Convex Hull - Monotone Chain

```

#include <stdio.h> // scanf, printf
#include <math.h> // sqrt
#include <algorithm> // sort
#include <utility> // pair

using namespace std;

#define X first
#define Y second
#define mp make_pair
#define pii pair<int,int>
#define llint long long
#define pll pair<llint, llint>
#define sq(x) ((x)*(x))
#define MAXN 100010

llint CCW(pll A, pll B, pll C) { // CCW = Counter-ClockWise.
    return A.X*B.Y-A.Y*B.X+B.X*C.Y-B.Y*C.X+C.X*A.Y-C.Y*A.X;
}

```

```

double dist(pll u, pll v) { //
    return sqrt(sq(u.X-v.X)+sq(u.Y-v.Y));
}

int N, hull[MAXN];
pii pt[MAXN];

int main() {
    scanf("%d", &N);
    for(int i=1; i<=N; ++i) {
        scanf("%d %d", &pt[i].X, &pt[i].Y);
    }
    sort(pt+1, pt+N+1);

    int top=0;
    for(int i=1; i<=N; ++i) { // Calculate upper envelope
        while(top>=2 && CCW(pt[hull[top-1]], pt[hull[top]],
            pt[i])>=0) {
            --top;
        }
        hull[++top]=i;
    }
    for(int i=N-1; i>=1; --i) { // Calculate lower envelope
        while(top>=2 && CCW(pt[hull[top-1]], pt[hull[top]],
            pt[i])>=0) {
            --top;
        }
        hull[++top]=i;
    }

    double length=0;
    printf("Convex Hull:\n");
    for(int i=1; i<top; ++i) {
        printf("%d %d\n", pt[hull[i]].X, pt[hull[i]].Y);
        length+=dist(pt[hull[i]], pt[hull[i+1]]);
    }
    printf("Convex Hull length: %lf\n", length);

    return 0;
}

```

10 DP Longest Common Increasing Subsequence - LCIS

```

#include <iostream>

```

```

using namespace std;

int a[100]={0};
int b[100]={0};
int f[100]={0};
int n=0, m=0;
int main(void)
{
    cin >> n;
    for (int i=1;i<=n;i++) cin >> a[i];
    cin >> m;
    for (int i=1;i<=m;i++) cin >> b[i];
    for (int i=1;i<=n;i++)
    {
        int k=0;
        for (int j=1;j<=m;j++)
        {if (a[i]>b[j] && f[j]>k) k=f[j];
            else if (a[i]==b[j] && k+1>f[j]) f[j]=k+1;
        }
    }
    int ans=0;
    for (int i=1;i<=m;i++)
        if (f[i]>ans) ans=f[i];
    cout << ans << endl;
    return 0;
}

```

11 DP Longest Common Subsequence - LCS

```

int dp[1001][1001];
int lcs(const string &s, const string &t)
{
    int m = s.size(), n = t.size();
    if (m == 0 || n == 0) return 0;
    for (int i=0; i<=m; ++i)
        dp[i][0] = 0;
    for (int j=1; j<=n; ++j)
        dp[0][j] = 0;
    for (int i=0; i<=m; ++i)
        for (int j=0; j<=n; ++j)
            if (s[i] == t[j])
                dp[i+1][j+1] = dp[i][j]+1;
            else
                dp[i+1][j+1] = max(dp[i+1][j], dp[i][j+1]);
    return dp[m][n];
}

```

12 DP Longest Increasing Subsequence - LIS

```
#include<iostream>
using namespace std;
int n=0;
int a[100]={0}, f[100]={0}, x[100]={0};
int main(void)
{
    cin >> n;
    for (int i=1;i<=n;i++)
    {
        cin >> a[i];
        x[i]=INT_MAX;
    }
    f[0]=0;
    int ans=0;
    for(int i=1;i<=n;i++)
    {
        int l=0, r=i;
        while (l+1<r)
        {
            int m=(l+r)/2;
            if (x[m]<a[i]) l=m; else r=m;
            // change to x[m]<=a[i] for non-decreasing case
        }
        f[i]=l+1;
        x[l+1]=a[i];
        if (f[i]>ans) ans=f[i];
    }
    cout << ans << endl;
    return 0;
}
```

13 Efficiencies

$$\begin{aligned}
 n \leq 10 &\rightarrow O(n!) \\
 n \leq 20 &\rightarrow O(2^n) \\
 n \leq 500 &\rightarrow O(n^3) \\
 n \leq 5000 &\rightarrow O(n^2) \\
 n \leq 10^6 &\rightarrow O(n \log n) \\
 n \text{ is large} &\rightarrow O(1), O(\log n)
 \end{aligned}$$

14 Exponential mod

```
long powmod(long base, long exp, long modulus) {
    base %= modulus;
    long result = 1;
    while (exp > 0) {
        if (exp & 1) result = (result * base) % modulus;
        base = (base * base) % modulus;
        exp >>= 1;
    }
    return result;
}
```

15 Formulas

$$\sum_{x=1}^n x = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$\sum_{x=1}^n x^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Geometric Progression:

$$a + ak + ak^2 + \dots + b = \frac{bk - a}{k - 1}$$

Harmonic:

$$\sum_{x=1}^n \frac{1}{x} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \leq \log_2(n) + 1$$

Logarithms:

$$\log_u(x) = \frac{\log_k(x)}{\log_k(u)}$$

Fermat's Theorem Generalization:

$$x^k \bmod m = x^{k \bmod (m-1)} \bmod m$$

16 Graph BFS

```
vector<int> adj[N];

queue<int> q;
bool visited[N];
int distance[N];

// Search starting at node x:
visited[x] = true;
distance[x] = 0;
q.push(x);
while (!q.empty()) {
    int s = q.front(); q.pop();
    // process node s
    for (auto u : adj[s]) {
        if (visited[u]) continue;
        visited[u] = true;
        distance[u] = distance[s]+1;
        q.push(u);
    }
}
```

17 Graph Bridges and Articulation Points

```
vector<int> gr[MAXN];
int num[MAXN], low[MAXN], par[MAXN];
int counter = 0;

void dfs(int u) {
    low[u] = num[u] = counter++;
    int kids = 0;

    for (auto v : gr[u]) {
        if (num[v] == -1) {
            // Here we have a forward edge.
            par[v] = u;
            dfs(v);
            kids++;

            // When u is root it doesn't have parents.
            // So par[u] must be -1.
            if (par[u] == -1 && kids > 1)
                printf("articulation point - %d\n", u);
            // For all other nodes
        }
    }
}
```

```

    if (par[u] != -1 && low[v] >= num[u])
        printf("articulation point - %d\n", u);

    if (low[v] > num[u])
        printf("bridge - (%d, %d)\n", u, v);

    low[u] = min(low[u], low[v]);
} else if (par[u] != v) {
    // Here we have a back edge.
    low[u] = min(low[u], num[v]);
}
}
}

// For multiple connected components:
memset(num, -1, sizeof num);
memset(low, -1, sizeof low);
memset(par, -1, sizeof par);
counter = 0;

// We number nodes starting from 1
for (int u = 1; u <= n; u++)
    if (num[u] == -1)
        dfs(u);

```

18 Graph DFS

```

vector<int> adj[N];
bool visited[N];

void dfs(int s) {
    if (visited[s]) return;
    visited[s] = true;
    // process node s
    for (auto u: adj[s]) {
        dfs(u);
    }
}

```

19 Graph Dijkstra

```

for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
q.push({0,x});
while (!q.empty()) {

```

```

    int a = q.top().second; q.pop();
    if (processed[a]) continue;
    processed[a] = true;
    for (auto u : adj[a]) {
        int b = u.first, w = u.second;
        if (distance[a]+w < distance[b]) {
            distance[b] = distance[a]+w;
            q.push({-distance[b],b});
        }
    }
}

```

20 Graph Eulerian Circuit

```

#include<iostream>
using namespace std;
int f[100]={0}, ans[100]={0};
bool g[100][100]={0}, v[100]={0};
int n=0, m=0, c=0;
void dfs(int k)
{
    for (int i=1;i<=n;i++)
        if (g[k][i])
        {
            g[k][i]=false;
            g[i][k]=false;
            dfs(i);
        }
    m++;
    ans[m]=k;
}
int main(void)
{
    cin >> n >> m;
    for (int i=1;i<=m;i++)
    {
        int x=0, y=0;
        g[x][y]=true;
        g[y][x]=true;
        f[x]++;
        f[y]++;
    }
    m=0;
    int k1=0;
    for (int i=1;i<=n;i++)
    {
        if (f[i]%2==1) k1++;
        if (k1>2)

```

```

    {
        cout << "error" << endl;
        return 0;
    }
    if (f[i]%2 && c==0) c=i;
}
if (c==0) c=1;
dfs(x);
for (int i=m;i>=1;i--) cout << ans[i] << endl;
return 0;
}

```

21 Graph Flood Fill

```

//component(i) denotes the
//component that node i is in
void flood_fill(new_component)
do
    num_visited = 0
    for all nodes i
        if component(i) = -2
            num_visited = num_visited + 1
            component(i) = new_component
    for all neighbors j of node i
        if component(j) = nil
            component(j) = -2
until num_visited = 0
void find_components()
num_components = 0
for all nodes i
    component(node i) = nil
for all nodes i
    if component(node i) is nil
        num_components = num_components + 1
        component(i) = -2
        flood_fill(component num_components)

```

22 Graph Flow-Cut, Ford-Fulkerson

```

int n;
vector<vector<int>> capacity;
vector<vector<int>> adj;

```

```

int bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();

        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][next]) {
                parent[next] = cur;
                int new_flow = min(flow, capacity[cur][next]);
                if (next == t)
                    return new_flow;
                q.push({next, new_flow});
            }
        }

        return 0;
    }

    int maxflow(int s, int t) {
        int flow = 0;
        vector<int> parent(n);
        int new_flow;

        while (new_flow = bfs(s, t, parent)) {
            flow += new_flow;
            int cur = t;
            while (cur != s) {
                int prev = parent[cur];
                capacity[prev][cur] -= new_flow;
                capacity[cur][prev] += new_flow;
                cur = prev;
            }

            return flow;
        }
    }
}

```

23 Graph Floyd Cycles

// Floyd's algorithm calculates cycle lengths in successor graphs

```

// Successor graph: we only have a single path that
// eventually ends in a cycle.

// The idea is to walk with two pointers until they meet,
// which happens at the cycle and how long it takes gives
// us the length of the cycle.

// Get the two pointers to meet eachother (cycle detected)
a = succ(x);
b = succ(succ(x));
while (a != b) {
    a = succ(a);
    b = succ(succ(b));
}
//Find First node of cycle
a = x;
while (a != b) {
    a = succ(a);
    b = succ(b);
}
first = a;
// Calculate Cycle Length
b = succ(a);
length = 1;
while (a != b) {
    b = succ(b);
    length++;
}

```

24 Graph Floyd-Warshall

```

// Shortest Path of All Pairs
// We construct distance matrix

// Initialization:
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == j) distance[i][j] = 0;
        else if (adj[i][j]) distance[i][j] = adj[i][j];
        else distance[i][j] = INF;
    }
}

// Use:
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            distance[i][j] = min(distance[i][j],

```

```

                distance[i][k]+distance[k][j]);
        }
    }
}

```

25 Graph Paths in a Grid

```

int sum[N][N];
for (int y = 1; y <= n; y++) {
    for (int x = 1; x <= n; x++) {
        sum[y][x] = max(sum[y][x-1], sum[y-1][x]) + value[y][x];
    }
}

```

26 Graph Prim - MST

```

int d[1001]={0};
bool v[1001]={0};
int a[1001][1001]={0};

int main(void)
{
    int n=0;
    cin >> n;
    for (int i=1;i<=n;i++)
    {
        int x=0, y=0, z=0;
        cin >> x >> y >> z;
        a[x][y]=z;
    }
    for (int i=1;i<=n;i++)
        for (int j=1;j<=n;j++)
            if (a[i][j]==0) a[i][j]=INT_MAX;
    cout << prim(1,n) << endl;
}

int prim(int u, int n)
{
    int mst=0,k;
    for (int i=0;i<d.length;i++) d[i]=INT_MAX;
    for (int i=0;i<v.length;i++) v[i]=false;
    d[u]=0;
    int i=u;
    while (i!=0)
    {

```

```

v[i]=true;k=0;
mst+=d[i];
for (int j=1;j<=n;j++)
    if (!v[j])
    {
        if (a[i][j]<d[j]) d[j]=a[i][j];
        if (d[j]<d[k]) k=j;
    }
i=k;
}
return mst;
}

```

27 Graph Representation

```

// The most common way to define graph is to use adjacency
// matrix
// example:
// (1) (2) (3) (4) (5)
// (1) 2 0 5 0 0
// (2) 4 2 0 0 1
// (3) 3 0 0 1 4
// (4) 6 9 0 0 0
// (5) 1 1 1 1 5
// its always a square matrix.
// suppose a graph has n nodes, if given exactly adjacency
// matrix
for (int i=1;i<=n;i++)
    for (int j=1;j<=n;j++)
    {
        cin << a[i][j] << endl;
    }
// Usually will go like this representation in data
// start_node end_node weight
// suppose m lines
for (int i=1;i<=m;i++)
{
    int x=0, y=0, t=0;
    cin >> x >> y >> t;
    a[x][y]=t;
    // if undirected graph
    a[y][x]=t;
}
// another variant: on the ith line, has data as
// end_node weight
// when you read data, you can assign matrix as
a[i][x]=t;
// if undirected graph

```

```

a[x][i]=t;
// Initialization of graph !!!IMPORTANT
// Depends on usage, normally initialize as 0 for all
// elements in matrix.
// so that 0 means no connection, non-0 means connection
// (for problem without weight, use weight as 1)
// If weights are important in this context (especially
// searching for path)
//
// Initialize graph as infinity for all elements in matrix.
//
// Another way to store graph is Adjacency list
// No space advantage if using array (unknown maximum number
// for in-degree).
// Big space advantage if using dynamic data structure (like
// list, vector).
// each row represent a node and its connectivity.
// we dont need it so much due to its search efficiency.
// lets define a node as
struct Node{
    int id; // node id
    int w; // weight
};
// suppose n nodes and m lines of inputs as
// start_node end_node weight
// assume using <vector> in this example
// g is a vector, and each element of g is also a vector of
// Node
for (int i=1;i<=m;i++)
{
    int x=0, y=0, t=0;
    cin >> x >> y >> t;
    Node temp; temp.id=y; temp.w=t;
    g[x].push_back(temp);
    // if undirected
    temp.id=x;
    g[y].push_back(temp);
}
// Note that you dont need this node structure if graph has
// only connectivity information.
//
//**** Special Structure ****/

// Special structure here is usually not a typical graph,
// like city-blocks, triangles
// They are represented in 2-d array and shows weights on
// nodes instead of edges.
// Note that in this case travel through edge has no cost,
// but visit node has cost.
//

```

```

// Triangles: Read data like this
// 1
// 1 2
// 4 2 7
// 7 3 1 5
// 6 2 9 4 6
for (int i=1;i<=n;i++)
    for (int j=i;j<=n;j++)
        cin >> a[i][j];
// Simple city-blocks: its just like first form of adjacency
// matrix, but this time
// represents weights on nodes, may not be square matrix.
// 1 2 4 5 6
// 2 4 5 1 3
// 4 5 2 3 6
for (int i=1;i<=n;i++)
    for (int j=1;j<=m;j++)
        cin >> a[i][j];
// More complex data structures: typical city-block
// structure may has some constraints on
// questions, but it has no boundaries. However, some
// questions requires to form a maze.
// In these cases, data structures can be very flexible, it
// totally depends on how the question
// presents the data. A usual way is to record its adjacent
// blocks information:
struct Block{
    bool l[4]; // if has 8 neighbors then use bool l[8];
    // label them as your favor, e.x.
    // 1 1 2 3
    // 4 x 2 8 x 4
    // 3 7 6 5
    // true if there is path, false if there is
    // boundary
    // other informations (optional)
    int weight;
    int component_id;
    // etc.
};
// Note that usually we use array from index 1 instead of 0
// because sometimes
// you need index 0 as your boundary, and start from index 1
// will give you
// advantage on locating nodes or positions

```

28 Graph SPFA - Better Bellman-Ford

```
int q[3001]={0}; // queue for node
int d[1001]={0}; // record shortest path from start to ith
node
bool f[1001]={0};
int a[1001][1001]={0}; // adjacency list
int w[1001][1001]={0}; // adjacency matrix

int main(void)
{
    int n=0, m=0;
    cin >> n >> m;
    for (int i=1;i<=m;i++)
    {int x=0, y=0, z=0;
        cin >> x >> y >> z; // node x to node y has weight z
        a[x][0]++;
        a[x][a[x][0]]=y;
        w[x][y]=z;
        /*
        // for undirected graph
        a[x][0]++;
        a[y][a[y][0]]=x;
        w[y][x]=z;
        */
    }
    int s=0, e=0;
    cin >> s >> e; // s: start, e: end
    SPFA(s);
    cout << d[e] << endl;
    return 0;
}

void SPFA(int v0)
{
    int t,h,u,v;
    for (int i=0;i<1001;i++) d[i]=INT_MAX;
    for (int i=0;i<1001;i++) f[i]=false;
    d[v0]=0;
    h=0; t=1; q[1]=v0; f[v0]=true;
    while (h!=t)
    {
        h++;
        if (h>3000) h=1;
        u=q[h];
        for (int j=1; j<=a[u][0];j++)
        {
            v=a[u][j];

```

```
            if (d[u]+w[u][v]<d[v]) // change to > if
                                // calculating longest path
            {
                d[v]=d[u]+w[u][v];
                if (!f[v])
                {
                    t++;
                    if (t>3000) t=1;
                    q[t]=v;
                    f[v]=true;
                }
            }
        }
        f[u]=false;
    }
}
```

29 Graph Topological Sorting - DFS

```
// Find any solution of topological sort.
#include<iostream>
using namespace std;

int f[100]={0}, ans[100]={0};
bool g[100][100]={0}, v[100]={0};
int n=0, m=0;

void dfs(int k)
{
    int i=0;
    v[k]=true;
    for (int i=1;i<=n;i++)
        if (g[k][i] && !v[i]) dfs(i);
    m++;
    ans[m]=k;
}

int main(void)
{
    cin >> n >> m;
    for (int i=1;i<=m;i++)
    {
        int x=0, y=0;
        cin >> x >> y;
        g[y][x]=true;
    }
}
```

```
m=0;
for (int i=1;i<=n;i++)
    if (!v[i]) dfs(i);
for (int i=1;i<=n;i++) cout << ans[i] << endl;
return 0;
}
```

30 Greatest Common Divisor - GCD

```
int gcd(int a, int b)
{
    if (b==0) return a;
    else return gcd(b, a%b);
}
```

31 Inverse Modulo

```
int invmod(int num, int mod) {
    return powmod(num, mod-2, mod);
}
```

32 Least Common Multiple - LCM

```
int lcm(int a, int b)
{
    return a*b/gcd(a,b);
}
```

33 Modulo Multiplication

```
// Computes (a * b) % mod (useful for REALLY big numbers
near limits)
long long mod_mul(long long a, long long b, long long mod) {
    long long x = 0, y = a % mod;
    while (b > 0) {
        if (b & 1)
            x = (x + y) % mod;
        y = (y * 2) % mod;
    }
}
```

```

    b /= 2;
}
return x % mod;
}

```

34 Primality Check

```

bool prime(int n)
{
    if (n<2) return false;if (n<=3) return true;
    if (!(n%2) || !(n%3)) return false;
    for (int i=5;i<=n;i+=6)
        if (!(n%i) || !(n%(i+2))) return false;
    return true;
}

```

35 Primes

```

// Under 100 there are 25 primes
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97

```

36 Quicksort

```

void quicksort(int *arr, int n) {
    if(n<=1) {
        return;
    }
    int m=rand()%n;
    swap(arr[0], arr[m]);
    m=0;
    for(int i=1; i<n; ++i) {
        if(arr[i]<arr[m]) {
            swap(arr[m+1], arr[i]);
            swap(arr[m], arr[m+1]);
            ++m;
        }
    }
    quicksort(arr, m);
    quicksort(arr+m+1, n-m-1);
}

```

37 Segment Trees Lazy

```

int seg[4*MAXN], // Segment Tree
    inpt[MAXN]; //

void Build(int ind=1, int start=0, int end=MAXN) {
    if(start == end) {
        seg[ind] = inpt[start];
        return;
    }

    int mid = (start+end)/2;
    Build(2*ind, start, mid);
    Build(2*ind+1, mid+1, end);

    seg[ind] = seg[2*ind] + seg[2*ind+1];
}

// We add lazy (so above build needs some changes...)

int seg[4*MAXN], lazy[4*MAXN];

void Update(int from, int to, int val, int ind=1, int start
=0, int end=MAXN) {
    if(start == from && end == to) {
        lazy[ind] += val;
        seg[ind] += val;
        return;
    }

    int mid = (start + end)/2;
    if(to <= mid) {
        Update(from, to, val, 2*ind, start, mid);
    }
    else if(from > mid) {
        Update(from, to, val, 2*ind+1, mid+1, end);
    }
    else {
        Update(from, mid, val, 2*ind, start, mid);
        Update(mid+1, to, val, 2*ind+1, mid+1, end);
    }
    seg[ind] = seg[2*ind] + seg[2*ind+1];
}

void Propagate(start, end, ind) {
    if(lazy[ind] == 0) {
        return;
    }

    int mid=(start+end)/2;

```

```

    Update(start, mid, lazy[ind], 2*ind, start, mid);
    Update(mid+1, to, lazy[ind], 2*ind+1, mid+1, end);
    lazy[ind] = 0;
}

int Query(int from, int to, int ind=1, int start=0, int end=
MAXN) {
    if(from==start && to==end) {
        return seg[ind];
    }

    Propagate(start, end, ind);

    int mid = (start + end)/2;
    if(to <= mid) {
        return Query(from, to, 2*ind, start, mid);
    }
    else if(from > mid) {
        return Query(from, to, 2*ind+1, mid+1, end);
    }
    else {
        return Query(from, mid, 2*ind, start, mid) + Query(
            mid+1, to, 2*ind+1, mid+1, end);
    }
}

```

38 Segment Trees

```

int tree[4*MAXN];

int sum(int a, int b, int k, int x, int y) {
    if (b < x || a > y) return 0;
    if (a <= x && y <= b) return tree[k];
    int d = (x+y)/2;
    return sum(a,b,2*k,x,d) + sum(a,b,2*k+1,d+1,y);
}

int sum(int a, int b) {
    return sum(a, b, 1, 0, n-1);
}

void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k]+tree[2*k+1];
    }
}

```

```
// Build using multiple add
// First initialized to all 0s
```

39 Template

```
// Compilation with: g++ -x c++ -g -O2 -std=gnu++20 -static
// {files}
#include <bits/stdc++.h>
using namespace std;

// consider to redefine data types to void data range
// problem
#define int long long // make everyone long long
#define double long double // make everyone long double
// function definitions
#undef int // main must return int
int main(void)
#define int long long // redefine int

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
}
```

40 Testcase Generator

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int N, MAXNUM;
    srand(time(NULL));
    scanf("%d %d", &N, &MAXNUM);
    for(int i=1; i<=N; i++) {
        printf("%d ", rand()%MAXNUM+1);
    }
    printf("\n");
    return 0;
}
```

41 Tree DFS

```
void dfs(int s, int e) {
    // process node s
    for (auto u : adj[s]) {
        if (u != e) dfs(u, s);
    }
}
```

42 Tree Depth Width

```
#include <iostream>
#include <queue>
#include <stack>
using namespace std;
int l[100]={0};
int r[100]={0};
stack<int> mystack;
int n=0;
int w=0;
int d=0;
int depth(int n)
{
    if (l[n]==0 && r[n]==0)
        return 1;
    int depthl=depth(l[n]);
    int depthr=depth(r[n]);
    int dep=depthl>depthr ? depthl:depthr;
    return dep+1;
}
void width(int n)
{
    if (n<=d)
    {
        int t=0,x;
        stack<int> tmpstack;
        while (!mystack.empty())
        {
            x=mystack.top();
            mystack.pop();
            if (x!=0)
            {
                t++;
                tmpstack.push(l[x]);
                tmpstack.push(r[x]);
            }
        }
    }
}
```

```
w=w>t?t:w;t;
mystack=tmpstack;
width(n+1);
}
}

int main(void)
{
    cin >> n;
    for (int i=1;i<=n;i++)
        cin >> l[i] >> r[i];
    d=depth(1);
    mystack.push(1);
    width(1);
    cout << w << " " << d << endl;
    return 0;
}
```

43 Tree Traversal

```
int L[100]={0};
int R[100]={0};

void DLR(int m)
{
    cout << m << " ";
    if (L[m]!=0) DLR(L[m]);
    if (R[m]!=0) DLR(R[m]);
}

void LDR(int m)
{
    if (L[m]!=0) LDR(L[m]);
    cout << m << " ";
    if (R[m]!=0) LDR(R[m]);
}

void LRD(int m)
{
    if (L[m]!=0) LRD(L[m]);
    if (R[m]!=0) LRD(R[m]);
    cout << m << " ";
}

int main(void)
{
    cin >> n;
    for (int i=1;i<=n;i++)
```

```

        cin >> L[i] >> R[i];
    (3)
    (4)DLR(1); cout << endl;
    LDR(1); cout << endl;
    LRD(1); cout << endl;
    return 0;
}

```

44 Tries

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

```

```

struct TrieNode
{
    // Array for child nodes of each node
    TrieNode* child[26];

    // for end of word
    bool wordEnd;

    TrieNode() {
        wordEnd = false;
        for (int i = 0; i < 26; i++) {
            child[i] = nullptr;
        }
    }
};

```

```

// Method to insert a key into the Trie
void insertKey(TrieNode* root, const string& key) {

```

```

    // Initialize the curr pointer with the root node
    TrieNode* curr = root;

```

```

    // Iterate across the length of the string
    for (char c : key) {

```

```

        // Check if the node exists for the
        // current character in the Trie
        if (curr->child[c - 'a'] == nullptr) {

```

```

            // If node for current character does
            // not exist then make a new node
            TrieNode* newNode = new TrieNode();

```

```

            // Keep the reference for the newly
            // created node
            curr->child[c - 'a'] = newNode;
        }

        // Move the curr pointer to the
        // newly created node
        curr = curr->child[c - 'a'];
    }

    // Mark the end of the word
    curr->wordEnd = true;
}

// Method to search a key in the Trie
bool searchKey(TrieNode* root, const string& key) {

    // Initialize the curr pointer with the root node
    TrieNode* curr = root;

    // Iterate across the length of the string
    for (char c : key) {

        // Check if the node exists for the
        // current character in the Trie
        if (curr->child[c - 'a'] == nullptr)
            return false;

        // Move the curr pointer to the
        // already existing node for the
        // current character
        curr = curr->child[c - 'a'];
    }

    // Return true if the word exists
    // and is marked as ending
    return curr->wordEnd;
}

int main()
{
    // Create an example Trie
    TrieNode* root = new TrieNode();
    vector<string> arr =
        {"and", "ant", "do", "geek", "dad", "ball"};
    for (const string& s : arr) {
        insertKey(root, s);
    }

    // One by one search strings
    vector<string> searchKeys = {"do", "geek", "bat"};

```

```

    for (string& s : searchKeys) {
        cout << "Key : " << s << "\n";
        if (searchKey(root, s))
            cout << "Present\n";
        else
            cout << "Not Present\n";
    }

    return 0;
}

```

45 Union Find

```

// Initialization
for (int i = 1; i <= n; i++) link[i] = i;
for (int i = 1; i <= n; i++) size[i] = 1;

```

```

int find(int x) {
    while (x != link[x]) x = link[x];
    return x;
}

```

```

bool same(int a, int b) {
    return find(a) == find(b);
}

```

```

void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (size[a] < size[b]) swap(a,b);
    size[a] += size[b];
    link[b] = a;
}

```

```

// Kruskal's use case
for (...) {
    if (!same(a,b)) unite(a,b);
}

```

46 cmath Tricks

```

// when the number is too large. use powl instead of pow.
// will provide you more accuracy.
powl(a, b)
(int)round(p, (1.0/n)) // nth root of p

```